## TOPIK 1

## *HELLO, PROGRAMMING !*

### A. Kenalan Yuk sama Algoritma

Saat kita akan belajar *programming*, tidak akan lepas dari kata “algoritma”, karena untuk memulai membuat sebuat program tentunya kita harus memahami terlebih dahulu bagaimana program kita akan berjalan, bagaimana alurnya, bagaimana proses didalam program dan apa hasil program kita. Lalu, apa sih aloritma itu ? **Algoritma adalah** urutan logis pengambilan keputusan untuk pemecahan sebuah masalah.

*Contoh algoritma :*

Terdapat 2 buah gelas yang berisi, gelas A berisi kopi berwarna hitam dan gelas B berisi susu berwarna putih. Kita ingin memindahkan isi gelas menjadi gelas A berisi susu dan gelas B berisi Kopi. Bagaimana cara kita memindahkan ?

Algoritma tukar gelas :

1. Tuangkan kopi dari gelas A ke gelas B
2. Tuangkan susu dari gelas B ke gelas A

Apakah algoritma tersebut benar ?

Ternyata, algoritma itu tidak benar ya teman-teman. Karena pada poin 1, jika kita menuangkan kopi dari gelas A ke gelas B yang masih berisi susu, maka akan bercampur menjadi kopi susu. Lalu, bagaimana solusinya ?

Yup! Kita harus menambahkan 1 gelas lagi yaitu gelas kosong, sebut saja gelas C. Jadi langkah algoritma yang logis adalah :

1. Tuangkan kopi dari gelas A ke gelas C (maka gelas A kosong)
2. Tuangkan susu dari gelas B ke gelas A (maka gelas B kosong)
3. Tuangkan kopi dari gelas C ke gelas A
4. Maka saat ini, gelas A berisi susu, gelas B berisi kopi dan gelas C kosong.

## B. Kenalan Yuk sama C#

Sebelumnya kita sudah membahas tentang algoritma, sekarang kita akan membahas tentang bahasa pemrograman C#. Tentunya dalam membuat program kita perlu mengenal bahasa pemrograman, banyak sekali bahasa pemrograman yang saat ini digunakan oleh para programmer, seperti C#, Phyton PHP, HTML, Java, Javacript, NodeJS, Golang dan lain sebagainya. Pada kelas pemrograman dasar kali ini, kita akan gunakan bahasa pemrograman C#.

**C# (dibaca *see sharp*) adalah** bahasa pemrograman yang dibuat oleh Microsoft dan ditargetkan berjalan di atas platform .NET (dotnet). Lalu, apa itu *dotnet* ? Dotnet itu semacam mesin virual yang tugasnya untuk menjalankan program C#, F#, VB.NET dan program lainnya. Selain itu, Dotnet juga menyediakan *tools, libary*, dan API yang kita butuhkan untuk membuat program C#. Bahasa C# biasa digunakan dalam pembuatan aplikasi berbasis desktop, contohnya aplikasi kasir, aplikasi parkir. Bahasa C# juga digunakan dalam pembuatan game, contohnya game *PokemonGO, Cuphead, Call of Duty,* dan *Monument Valley.*

## C. Lets Code !

Selanjutnya, sebelum belajar coding, kita harus mempersiapkan alatnya yaitu Visual Studio (download link : https://visualstudio.microsoft.com/) atau bisa praktik online menggunakan https://www.programiz.com/csharp-programming/online-compiler/ .

Bagaimana cara installasi nya ? kita bahas di pertemuan pertama ya teman-teman. Sampai jumpa !

## D. HELLO WORLD!;

### 1) Struktur Dasar C#

Struktur program C# yang paling dasar, terdiri dari tiga bagian:

```
//1.deklarasi pustaka
using System;

//2.deklarasi class
public class HelloWorld
{
    //3.deklarasi fungsi
    public static void Main(string[] args)
    {
       Console.Write("Hello World !");

    }
}
```

a. Bagian deklarasi Pustaka

Ini adalah bagian paling atas dari program C#. Pada bagian ini, kita menuliskan pustaka (*library)* yang dibutuhkan dalam program. Apa itu *library*? *Library* berisi sekumpulan fungsi, *method, class*, objek, konstanta, dan *variabel* yang bisa kita gunakan ulang di dalam program.

b. Bagian *Class*

*Class* berperan sebagai nama program.

c. Bagian Fungsi atau Method

Fungsi adalah jantung dari program yang harus ada di dalam setiap program adalah fungsi Main(). Kalau tidak ada fungsi ini, program tidak akan bisa dijalankan. Karena fungsi Main() merupakan fungsi utama yang akan dieksekusi pertama kali.

### 2) Penulisan Statement dan Ekspresi

Penulisan statemen dan ekspresi dalam C# harus diakhiri dengan titik koma (;) atau sering disebut *semicolon*. Jika tidak, maka akan terjadi *error.*

```
    Console.Write("Hello World !");
```

### 3) Penulisan Blok Kode

Blok kode di dalam C# dibungkus menggunakan kurung kurawal { ... }. Biasanya blok kode digunakan untuk membungkus lebih dari satu

statement. Jika statement hanya ada satu baris, maka bisa kita tidak bungkus dengan tanda kurung kurawal.

```
//1.deklarasi pustaka
using System;

//2.deklarasi class
public class HelloWorld
{
    //3.deklarasi fungsi
    public static void Main(string[] args)
    {
        Console.Write("Hello World !");

    }
}
```

4) **Penulisan Komentar**

Komentar adalah teks yang tidak akan dieksekusi. Biasanya digunakan untuk menuliskan keterangan dan menon-aktifkan sebuah kode. Komentar dalam C# ditulis dengan garis miring ganda dan garis miring bintang.

```
//ini adalah contoh penulisan komentar

/*Komentar DilesinAcademy
...
...
*/
```

5) **Gaya Case**

Bahasa pemrograman C# menggunakan Pascal Case dan Camel Case.

- Untuk penulisan nama *Class* dan Fungsi atau *Method*, selalu diawali dengan huruf kapital.
- Apabila nama fungsi dan class terdiri dari dua suku kata, maka penulisannya digabung (tanpa spasi).
- Nama variabel ditulis dengan huruf kecil didepan

6) **Penulisan String dan Angka**

String atau teks, biasanya ditulis dengan diapit tanda petik. Sedangkan angka ditulis tanpa tanda petik.

### 7) Reserved Keyword

Reserved Keywords adalah kata kunci yang tidak boleh digunakan sebagai nama variabel. Pada C# terdapat reserved keyword antara lain :

```
abstract  as  base  bool
break  byte  case  catch
char  checked  class  const
continue  decimal  default  delegate
do  double  else  enum
event  explicit  extern  false
finally  fixed  float  for
foreach  goto  if  implicit
in  int  interface  internal
is  lock  long  namespace
new  null  object  operator
out  override  params  private
protected  public  readonly  ref
return  sbyte  sealed  short
sizeof  stackalloc  static  string
struct  switch  this  throw
true  try  typeof  uint
ulong  unchecked  unsafe  ushort
using  using static  virtual  void
volatile  while
```

## E. OUTPUT - INPUT

### 1) Output

Output artinya mencetak informasi yang dihasilkan oleh program. Sebagai contoh kita akan menampilkan tulisan pada program. Pada C# kita bisa menuliskan perintah output seperti berikut :

```
using System;

public class HelloWorld
{
    public static void Main(string[] args)
    {
       Console.Write("Selamat datang di Dilesin Academy");

    }
}
```

**2) Input**

Input artinya meminta data dari pengguna program atau ketika pengguna program harus mengisi data pada program. Sebagai contoh kita meminta pengguna untuk menulis data nama pada program kita. Pada C# kita bisa menuliskan perintah input seperti berikut :

```
using System;

public class HelloWorld
{
    public static void Main(string[] args)
    {
        Console.Write("Tuliskan nama kamu: ");
        string nama = Console.ReadLine();
        Console.WriteLine("Hi, {0} selamat datang!", nama);

    }
}
```

Setelah kamu praktikan sintaks diatas terdapat “write” dan “WriteLine”, apa perbedaannya ?

## Topik 1 – Practice 1

Ayo buat program dengan tampilan seperti berikut !

```
Tuliskan nama kamu             : Milea
Tuliskan usia kamu             : 17
Tuliskan kota kamu tinggal     : Purwokerto
Tuliskan nama sekolah / kerjamu: SMA Dilesin Academy
Tuliskan alasanmu mengikuti kelas ini:
Karena ingin menambah ilmu dan ketrampilan tentang coding
------------------------------------------------------------

Hallo DilesinAcademy! Perkenalkan nama saya Milea usia saya 17 tahun.
    Saya tinggal di Purwokerto. Saat ini saya bersekolah di SMA Dilesin
    Academy. Alasan saya mengikuti kelas dasar pemrograman adalah
    Karena ingin menambah ilmu dan ketrampilan tentang coding
```

## TOPIK 2

# TIPE DATA, VARIABEL, KONSTANTA

## A. TIPE DATA

Tipe Data adalah format penyimpanan data.

Berikut ini beberapa jenis tipe data yang sering digunakan:

- *string* adalah tipe data yang berupa teks, contoh: "Saya hebat!";
- *int* adalah tipe data yang berupa angka, Contoh: 3, 9, 0;
- *float* adalah tipe data yang berupa angka pecahan, contoh 1.2f, 2.4f, 5.5f (huruf f artinya float);
- *bool* adalah tipe data boolean yang hanya berisi true dan false.

## B. VARIABEL

Apa itu Variabel ?

Saat kita menggunakan fungsi input, kita membutuhkan variabel untuk menyimpan data yang dimasukan ke program. Variabel adalah sebuah wadah penyimpanan data pada program yang akan digunakan selama program itu berjalan. Apakah kita akan bisa membuat program tanpa menggunakan variabel ? Bisa, tapi hanya program semacam *Hello World*. Untuk membuat program yang lebih kompleks, kita membutuhkan bantuan variabel.

Ada dua cara membuat variabel di C#:

- Pertama (eksplisit), dengan menuliskan tipe data lalu diikuti nama variabel.

```
// membuat variabel kosong
    string alamat;
    int umur;
    float beratBadan;
    bool isMenikah;

// membuat variabel dan langsung mengisinya
    string nama = "DilesinAcademy";
    int umur = 18;
    float beratBadan = 59.34;
```

- Kedua (implisit), dengan tidak menuliskan tipe data

```
var namaVariabel = "isi variabel";
```

Pembuatan variabel dengan *var* harus kita isi nilainya, karena kalau tidak.. akan terjadi error seperti ini:

```
An implicitly typed local variable declarator must include an initializer
```

Aturan membuat variabel :

a. Jangan menggunakan angka di awal nama variabel.
b. Jangan menggunakan simbol.
c. Jangan menggunakan nama dengan kata kunci yang sudah ada di C#
d. Dianjurkan menggunakan nama variabel yang mewakili dari isi variabel.
e. Nama variabel bersifat case sensitive. Artinya huruf besar dan kecil dibedakan.

## C. KONSTANTA

Konstanta hampir sama seperti variabel, bedanya konstanta bersifat *immutable* artinya nilainya tidak bisa diisi ulang alias konstan. Contoh pembuatan konstanta:

```
// membuat konstanta
    const double Phi = 3.14;

// mengisi ulang (jangan pernah lakukan ini)
    Phi = 5.32
```

Konstanta tidak bisa diisi ulang, maka ketika kita mencoba mengisi ulang konstanta, maka akan *error*.

### Topik 2 – Practice 1

Setelah mempelajari materi tentang input output, tipe data, variabel dan konstanta. Buatlah program untuk menghitung :

a. Luas Persegi Panjang
b. Luas Lingkaran

## TOPIK 3
## OPERATOR

Apa itu operator ?

Operator adalah sebuah simbol. Simbol yang digunakan untuk melakukan operasi tertentu. Misalnya: Kita ingin menjumlahkan nilai dari variabel x dan y, maka kita bisa menggunakan operator penjumlahan (+).

Jenis-jenis operator :

a. **Operator Artimatika**

   Operator aritmatika adalah operator untuk melakukan operasi aritmatika seperti kali, bagi, tambah, kurang.

| Nama Operator | Simbol |
|---|---|
| Penjumlahan | + |
| Pengurangan | - |
| Perkalian | * |
| Pembagian | / |
| Sisa Bagi | % |
| Increment | ++ |
| Decrement | -- |

Contoh Penulisan Operator Artimatika :

```
4  using System;
5
6  public class Operator_Aritmatika
7  {
8      public static void Main(string[] args)
9      {
10     int jeruk, pear, hasil = 0;
11
12     Console.Write("jeruk = ");
13     jeruk = int.Parse(Console.ReadLine());
14     Console.Write("pear = ");
15     pear = int.Parse(Console.ReadLine());
16
17     // operasi penjumlahan dengan operator +
18     hasil = jeruk + pear;
19     Console.WriteLine ($"Hasil jeruk + pear = {hasil}");
20
21     }
22 }
```

Setelah menuliskan sintaks diatas, apa hasilnya ?

Operasi penjumlahan dapat kita lakukan untuk tipe data numerik atau angka. Jika kita melakukan penjumlahan pada tipe data string, maka yang terjadi adalah penggabungan.bukan penjumlahan.

Contoh :

string nama = "Dilesin" + "Academy"

maka variabel nama akan berisi “DilesinAcademy”

## Topik 3 – Practice 1

Ayo kita praktikan keseluruhan Operator Aritmatika pada sebuah program sederhana, dengan ketentuan sebagai berikut :

- Terdapat 2 variabel bertipe integer dengan nama A dan B
- Program menampilkan hasil dari :

  A + B  A x B  sisa hasil A : B

  A – B  A : B  increment A dan decrement B

**b. Operator Penugasan**

Operator penugasan (*Assignment Operator*) merupakan operator untuk meberikan tugas pada variabel. Biasanya digunakan untuk mengisi nilai.

| Nama Operator | Sombol |
|---|---|
| Pengisian Nilai | `=` |
| Pengisian dan Penambahan | `+=` |
| Pengisian dan Pengurangan | `-=` |
| Pengisian dan Perkalian | `*=` |
| Pengisian dan Pembagian | `/=` |
| Pengisian dan Sisa bagi | `%=` |
| Pengisian dan shift left | `<<=` |
| Pengisian dan shift right | `>>=` |

Contoh Penulisan Operator Penugasan :

```
public static void Main(string[] args)
{
    // menggunakan operator = untuk mengisi nilai
    int jeruk = 10;
    int pear = 8;
    // mengisi ulang nilai variabel mangga
    jeruk = 15;
    Console.WriteLine($"jeruk = {jeruk}");
    // menggunakan += untuk mengisi dan menjumlahkan
    pear += 6;
    Console.WriteLine($"pear = {pear}");
}
```

Bagaimana hasil sintaks diatas ?

Pembahasan :

- Pada contoh tersebut, kita mengisi ulang nilai variabel jeruk menjadi 15 dengan operator pengisian (=).

- Lalu untuk variabel pear kita menambahkannya dengan 6.
  Penulisan  pear+=6; adalah bentuk sederhana dari pear = pear + 6;

## Topik 3 – Practice 2

Ayo kita praktikan keseluruhan Operator Penugasan pada sebuah program sederhana, dengan ketentuan :

- Terdapat 2 variabel bertipe integer dengan nama_var(nilai ) = Jeruk (10) dan Pear(8)
- Jeruk mendapat operator :
  Pengisian dan penambahan (+2)
  Pengisian dan pengurangan (-2)
  Pengisian dan shift left
- Pear mendapat operator
  Pengisian dan perkalian (x2)
  Pengisian dan pembagian (:2)
  Pengisian dan shift right
- Praktikan juga operator Pengisian dan Sisa bagi pada kedua variabel

**c. Operator Perbandingan**

Operator pembanding adalah operator untuk membandingkan dua buah nilai. Operator ini juga dikenal dengan operator relasi.

| Nama Operator | Simbol |
|---|---|
| Lebih Besar | > |
| Lebih Kecil | < |
| Sama Dengan | == |
| Tidak Sama dengan | != |
| Lebih Besar Sama dengan | >= |
| Lebih Kecil Sama dengan | <= |

Nilai yang dihasilkan dari operasi pembanding akan berupa boolean *True* dan *False.*

Contoh penulisan Operator Perbandingan :

```
using System;

public class HelloWorld
{
    public static void Main(string[] args)
    {
        int mangga, apel;

        Console.Write("jumlah mangga = ");
        mangga = int.Parse(Console.ReadLine());
        Console.Write("jumlah apel = ");
        apel = int.Parse(Console.ReadLine());

        Console.WriteLine("Hasil perbandingan: ");
        Console.WriteLine($"mangga > apel : {mangga > apel}");
    }
}
```

Bagaimana hasil sintaks diatas ?

Operator perbandingan nanti akan banyak kita pakai untuk membuat logika program di materi percabangan. Sebagai contoh, coba tambahkan sintaks berikut pada sintaks sebelumnya.

```
if ( mangga > apel )
{
    Console.WriteLine("Mangga lebih banyak");
}
```

## Topik 3 – Practice 3

Ai akan membuat sebuah program yang dapat membandingkan berat dari buah-buahan. Sebut saja buah A dan buah B . Bantulah ai membuat program perbandingan tersebut, dimana pada program tersebut akan mengetahui apakah :

- buah A lebih besar dari buah B
- buah A lebih kecil dari buah B
- buah A sama dengan buah B
- buah A tidak sama dengan buah B
- buah A lebih besar sama dengan dari buah B
- buah A lebih kecil sama dengan dari buah B

**d. Operator Logika**

Kalau kamu pernah belajar logika matematika, pasti tidak akan asing dengan operator ini. Hasil operasi dari operator logika sama seperti operator perbandingan, yakni boolean True dan False.

| Nama Operator | Simbol |
|---|---|
| Logika AND | `&&` |
| Logika OR | `\|\|` |
| Negasi/kebalikan | `!` |

Yang perlu diingat adalah :

***Operan harus bertipe boolean.*** Karena jika tidak, maka akan terjadi *error.*

Operator Logika digunakan untuk membuat operasi logika.

Misalnya seperti ini:

Pernyataan 1: Mangga rasanya manis

Pernyataan 2: Jeruk rasanya manis

Jika kita coba buktikan dengan tabel kebenaran logika, maka bisa kita buat seperti ini :

- Tabel Kebenaran Logika AND

| Pernyatan 1 | Pernytaan 2 | Hasil Logika AND |
|---|---|---|
| `true` | `true` | `true` |
| `true` | `false` | `false` |
| `false` | `true` | `false` |
| `false` | `false` | `false` |

- Tabel Kebenaran Logika NOT

| Pernyatan 1 | Hasil Logika NOT |
|---|---|
| `true` | `false` |
| `false` | `true` |

- Tabel Kebenaran Logika OR

| Pernyatan 1 | Pernytaan 2 | Hasil Logika OR |
|---|---|---|
| `true` | `true` | `true` |
| `true` | `false` | `true` |
| `false` | `true` | `true` |
| `false` | `false` | `false` |



# TOPIK 4
# PERCABANGAN

Percabangan adalah cara yang digunakan untuk mengambil keputusan apabila di dalam program dihadapkan pada beberapa kondisi tertentu. Untuk menggunakan fungsi percabangan kita bisa menggunakan perintah IF…ELSE atau SWITCH.

**a. IF…ELSE**

Statement IF digunakan untuk mengeksekusi beberapa kode program apabila mempunyai kondisi True atau False

```
if (condition) {
    //statements
}
```

Statement IF menentukan kondisi ekspresi yang akan dievaluasi. Apabila kondisi benar, pernyataan dalam kurung kurawal "{}" akan dieksekusi. Apabila kondisi salah, maka akan di abaikan Kemudian komputer akan melanjutkan program yang berada setelah tubuh dari statement IF tersebut.

Contoh :

```
using System;

public class Program
{
    public static void Main()
    {
            var x = 30;
            var y = 83;

            if (y > x)
            {
               Console.WriteLine("y Lebih Besar daripada x");
            }
    }
}
```

Pada kode Program di atas, IF menjelaskan bahwa Apabila 83 lebih dari 30 maka statement Console.WriteLine akan di jalankan yang mana menampilkan kata "y Lebih Besar daripada x" pada layar. Lalu bagaimana jika nilai yang dimasukan y tidak lebih besar dari x, apa yang akan ditampilkan ?

Ya betul! Karena kodisi IF tidak terpenuhi maka tidak ada yang ditampilkan. Oleh karena itu kita membutuhkan statement ELSE agar percabangan bisa memiliki 2 kondisi atau lebih.

Statement Else adalah pengikut dari Statement IF, apabila Statement IF adalah statement Utama maka statement Else adalah statement kedua atau statement Opsional. Statement Else akan di eksekusi apabila Statement IF tidak mendapatkan hasil atau mempunyai hasil Salah.

```
if (condition) {
  //statements
}
else {
 //statements
}
```

Compiler akan menguji kondisi:

- Apabila statement bernilai **true**, maka kode di dalam pernyataan **IF** akan dieksekusi.
- Apabila statement bernilai **false**, maka kode dalam pernyataan **Else** akan dieksekusi.

Contoh Program :

```
1  using System;
2  public class CekNilai
3  {
4      public static void Main(string[] args)
5      {
6          int nilai;
7          Console.WriteLine("Masukan nilai kamu: ");
8          nilai = int.Parse(Console.ReadLine());
9
10         if (nilai< 75) {
11             Console.WriteLine("Kamu Gagal");
12         }
13
14         else {
15             Console.WriteLine("Kamu Berhasil");
16         }
17     }
18 }
```

## Topik 4 – Practice 1

Buatlah sebuah program sederhana untuk melakukan pengecekan nilai dengan menggunakan statement IF..ELSE, dengan ketentuan :

- Jika nilai 90 – 100 akan muncul predikat A
- Jika nilai 80 – 89 akan muncul predikat B
- Jika nilai 70 – 79 akan muncul predikat A
- Jika nilai 60 – 69 akan muncul predikat D
- Jika nilai 50 – 59 akan muncul predikat E
- Jika nilai < 50 akan muncul Anda Tidak Lulus

b. SWITCH…CASE

Secara konsep SWITCH…CASE sama dengan IF…ELSE, hanya saja secara penulisan lebih rapi dah juga digunakan pada kasus tertentu. Statement Switch memastikan persamaan antara, nilai 1 dengan nilai yang ada pada case didalam Switch. Apabila nilai sama, maka operasi akan di eksekusi.

```
switch (expression) {
  case nilai1:
    statement(s);
    break;
  case nilai2:
    statement(s);
    break;
    ...
  case nilaiN:
    statement(s);
    break;
}
```

Didalam sebuah pernyataan Switch terdapat beberapa pernyataan case,apabila variabel yang dituju sama nilainya dengan nilai case maka pernyataan dalam case akan di eksekusi.

Contoh Program Switch..Case

```
2  public class CekUsia
3  {
4      public static void Main(string[] args)
5      {
6        int age = 25;
7          switch (age) {
8              case 10:
9              Console.WriteLine("Anak-Anak");
10             break;
11             case 17:
12             Console.WriteLine("Remaja");
13             break;
14             default:
15             Console.WriteLine("Tak Di ketahui");
16             break;
17         }
18     }
19 }
```

Perhatikan contoh diatas, kasus diatas memiliki fungsi untuk membandingkan usia. Namun, statement switch…case adalah pilihan yang tidak tepat, karena pada switch case tidak bisa menggunakan operator pembanding, sehingga nilai yang terbaca hanya akan dinilai sama atau tidak sama dengan nilai yang ada, jadi kita tidak bisa menerapkan range usia.

Saat menggunakan pernyataan Switch jangan lupa menambahkan ***break*** pada akhir case**.** Didalam pernyataan Switch terdapat kata ***default***, nah apakah default itu? sama halnya dengan *Else* pada statement *IF*, *default* dapat difungsikan sebagai penolakan apabila nilai variabel tidak ada yang sama dengan *Case* pada statement *Switch*.

## Topik 4 – Practice 2

Buatlah sebuah program sederhana untuk menampilkan nama bulan dengan menggunakan statement SWITCH…CASE, dengan ketentuan jika kita menginputkan angka 1 maka akan muncul Januari, angka 2 maka Februari, dst.

## TOPIK 5
## PERULANGAN

Perulangan merupakan suatu proses dimana kita dapat mengulang satu atau beberapa perintah atau eksekusi blok kode tanpa henti, selama kondisi yang dijadikan acuan terpenuhi. Pada perulangan kita bisa menggunakan statement WHILE, DO..WHILE dan FOR.

### a. WHILE

Perulangan While berulang kali mengeksekusi statement atau pernyataan sampai dengan suatu kondisi yang ditetapkan telah terpenuhi. berikut merupakan susunan kode dari Perulangan While.

```
while (condition) {
    statement(s);
}
```

Tubuh Perulangan pada perulangan while adalah kumpulan blok statement yang berada di dalam kurung kurawal program. Pengulangan beriterasi saat kondisi bernilai benar. Pada saat kondisi bernilai salah, kontrol program akan di alihkan ke baris yang mengikuti perulangan tersebut.

Contoh program WHILE

```
using System;
public class PerulanganWhile
{
    public static void Main(string[] args)
    {
     int nilai= 1;
        while (nilai < 6) {
        Console.WriteLine(nilai);
        nilai = nilai + 1;
        }
    }
}
```

- Pada contoh di atas nilai variabel ditetapkan terlebih dahulu yaitu, int nilai yang bernilai 1.
- Kemudian Perulangan while akan memastikan kondisi, yang mana kondisi tersebut adalah ( nilai < 6 ), setelah itu program akan mengeksekusi statement yang ada pada tubuh perulangan, dengan increment nilai dari variabel nilai yaitu bertambah 1 setiap pengulangan berjalan.
- Setelah pengulangan ke 5 maka nilai pada variabel nilai akan bernilai 6, dan kondisi menjadi bernilai false atau salah. saat hal tersebut terjadi maka perulangan akan berhenti.
- Nilai Increment pada perulangan while dapat di rubah. apa bila nilai inceament di rubah maka jumlah pengulangan pada perulangan while akan berubah juga.

### Topik 5 – Practice 1

Dengan menggunakan perulangan WHILE, buatlah sebuah program sederhana dimana pengguna dapat mengisikan data berupa angka sebanyak 4 kali , dan akan menghasilkan jumlah total angka yang diinputkan. Sebagai contoh :

```
Isi Nilai : 10
Isi Nilai : 20
Isi Nilai : 30
Isi Nilai : 40
Total Nilai Adalah = 100
```

**b. DO..WHILE**

Perulangan do while hampir mirip dengan perulangan while. Perulangan do while mengecek kondisi di akhir statement perulangan.

```
do {
   statement(s);
} while (condition);
```

Contoh perulahan DO…WHILE

```
using System;

public class PerulanganDoWhile
{
    public static void Main(string[] args)
    {
       int a = 0;
        do {
            Console.WriteLine(a);
            a++;
        } while(a < 5);
    }
}
```

- Pada kode di atas variabel ‘a’ kita inisialisasikan dahulu dengan nilai 0, mengapa 0 ?, karena kita akan membuat program yang menampilkan angka mulai dari 0 sampai dengan 5.
- Setelah itu kita gunakan perulangan do while, yang artinya do atau lakukan, tampilkan a, a++ while atau saat ‘a’ kurang dari atau sama dengan 5.

## Topik 5 – Practice 2

Praktikan kode program berikut !

```
using System;

public class PerulanganDoWhile
{
    public static void Main(string[] args)
    {
     int a = 42;
        do {
        Console.WriteLine(a);
        } while (a > 0);
    }
}
```

Jelaskan yang terjadi pada perulangan tersebut !

## c. FOR

Perulangan For, berbeda dengan yang lain, perulangan For lebih efisien dalam penulisan programnya.

```
for ( init; condition; increment ) {
   statement(s);
}
```

- Init akan di eksekusi terlebih dahulu, dan tidak akan di ulang, karena init berarti initialization atau inisialisasi variabel dalam perulangan.
- Condition berisi kondisi yang digunakan untuk mengevaluasi perulangan, apakah kondisi untuk mengulang blok kode masih terpenuhi atau tidak. Biasa berisi operator perbandingan
- Increment berfungsi untuk mengatur jumlah perulangan.
- Jika kondisi terpenuhi maka, tubuh perulangan (statement) akan dijalankan. Jika tidak maka perulangan akan berhenti.

Contoh program FOR

```
using System;

public class HelloWorld
{
    public static void Main(string[] args)
    {
        for (int a = 0; a <= 10; a++) {

            Console.WriteLine(a);
    }
  }
}
```

- Pada init kita deklarasikan dengan Variabel ‘a’ dan mengatur nilainya dengan angka 0.
- a < 10 merupakan Kondisi.
- setelah Iterasi, increment ( a++ ) akan di eksekusi.
- Saat a bernilai 11 maka kondisi menjadi salah, dan pengulangan akan berhenti.

## Topik 5 – Practice 3

Dengan menggunakan perulangan FOR, buatlah sebuah program sederhana dimana akan menghitung mundur dari angka 100 sampai dengan angka 1 dengan selisih 3 angka !



# TOPIK 6
# ARRAY

**Apa itu Array ?**

Array merupakan variabel yang dapat menampung banyak nilai sekaligus. Array dideklarasikan dengan tipe data diikuti dengan sepasang kurung siku. Penerapan Array sering dilakukan jika seseorang harus menyimpan banyak data dalam satu tipe data, ambil contoh misalkan kita mempunyai 5 data tentang pengukuran tinggi badan, dan kita ingin masukkan itu ke dalam program yang kita buat, jika tanpa Array maka kita akan buat kode programnya seperti :

```
int tinggiA = 156;
int tinggiB = 159;
int tinggiC = 154;
int tinggiD = 160;
int tinggiE = 161;
```

Sedangkan jika kita memakai Array maka kode program kita akan lebih sederhana dan enak dilihat, seperti halnya pada kode berikut.

```
int[ ] tinggi = new int [5] {

     156,159,154,160,161
};
```

Bagaimana menggunakan Array ?

Apabila kamu mau menggunakan Array , maka ada aturan dasar yang harus kamu ketahui yaitu untuk mendeklarasikan variabel Array harus ditulis dengan cara :

tipedata[ ] variabelmu contohya string[ ] var_tinggi

- *Assignment*

  Assignment adalah proses memasukkan data ke dalam Array dan selain itu dilakukan juga pemberian ukuran Array

- *Accessing*

  Accessing adalah proses atau cara untuk mengakses data dalam Array , namun yang harus diingat adalah awalan atau elemen pertama dari Array bukanlah indek ke 1 melainkan indek ke 0

  Contoh program Array

```
using System;

public class contoh_array
{
    public static void Main(string[] args)
    {
        //membuat variabel array
      string[ ] Marga = new string [4]
    {
        //memberi nilai pada variabel array
    "Kim", "Park", "Lee", "Han"
    };
    //mengakses dan menampilkan variabel array
    string Marga1 = Marga[1];
    Console.WriteLine(Marga1);
  }
}
```

Sintaks diatas akan menampilkan 1 variabel array. Pastilah mudah jika isi dari variabel hanya berisi 4 data seperti diatas. Bagaimana jika ada 1000 data atau bahkan 100.000 data ?

**Topik 6 – Practice 1**

Buatlah sebuah array yang berisi nama-nama bulan. Selanjutnya tampilkan :

- Tampilkan bulan November saja
- Tampilkan semua bulan, tanpa harus menuliskan perintah output satu per satu

**Apa itu Array Multidimensi ?**

Bentuk paling sederhana dari Array multidimensi adalah Array 2 Dimensi, Array 2 dimensi merupakan gabungan dari 2 Array 1 Dimensi. Array 2 dimensi dapat juga digambarkan seperti sebuah tabel data, yang memiliki beberapa baris dan kolom. Sebagai Contoh, kita akan membuat array dengan ukuran 2 dimensi 2 x 3 dengan tipe data integer.

```
int[ , ] x = new int[2,3];
```

Hasil :

| | Column 1 | Column 2 | Column 3 |
|---|---|---|---|
| Row 1 | X[0][0] | X[0][1] | X[0][2] |
| Row 2 | X[1][0] | X[1][1] | X[1][2] |

Selain dengan cara diatas, kamu juga bisa menginisialisasi array multidimensi dengan cara yang sama, dengan array satu dimensi.

```
int[ , ] Arr = { {2, 8}, {5, 7}, {9, 6} };
```

Contoh program Array Multidimensi :

```
using System;

public class contoh_array
{
    public static void Main(string[] args)
    {
    //inisialisasi array 2 dimensi
       int[ , ] angka = {{2, 3}, {4, 5}};

    // mengakses elemen dari baris ke-1
    Console.WriteLine("Element at index [0, 0] : "+numbers[0,0]
        );

    // mengakses elemen dari baris ke-2
       Console.WriteLine("Element at index [1, 0] : "+numbers[1
           ,0]);
    }
}
```

## Topik 6 – Practice 2

Setelah mempelajari cara menampilkan seluruh nilai array, tampilkan keseluruhan nilai array berdasarkan Row / barisnya, dari deklarasi array berikut !

```
using System;

public class contoh_array
{
    public static void Main(string[] args)
    {
   int[ , ] numbers = { {2, 3, 9}, {4, 5, 9} };

    }
}
```

**OUTPUT**

```
Row 0: 2 3 9
Row 1: 4 5 9
```

## TOPIK 7
## FUNCTION (FUNGSI)

Sebuah fungsi memungkinkan kamu untuk meringkas sepenggal kode dan memangil kode tersebut dari bagian lain kodemu. Kamu mungkin akan segera menemukan situasi dimana kamu harus mengulang sebuah kode, dari berbagai bagian, dan disinilah peran penting sebuah fungsi.

Struktur fungsi pada C#

```
<visibility> <return type> <name>(<parameters>)
{
    <function code>
}
```